Lab1:Verilog语法实验报告

一、实验目标

1. 学习并掌握基本的Verilog语法知识；
2. 能够阅读并且编写一些简单的Verilog代码；
3. 能够将Verilog语言与硬件电路对应起来，理解硬件描述语言(HDL)的内涵。

二、实验内容

1. 学习 Verilog 语言的基本语法，了解 Verilog 语言的特性。本次实验介绍了 Verilog 的语句的一些基本组成，如数值系统、标识符、变量类型、运算符等，也介绍了 Verilog 的赋值语句、条件语句，同时还讲述 Verilog 模块的相关知识，包括模块的结构、模块的例化、参数的传递等。实验还介绍了 Verilog 的三种描述方式：结构化描述、数据流描述、行为级描述，以及时钟信号、复位信号的基本概念与实现操作。
2. 配置本地 VSCode 开发环境。根据实验教程，配置好 VSCode 环境，具有语法高亮、变量识别、语句检查等功能，并将 VSCode 设置为 Vivado 的默认编辑器。
3. 完成 Lab1 的相关实验练习题。

三、实验练习解答

【必做部分】

题目 1：向量翻转

由于本题向量的长度已经固定为8位，则使用拼接运算符，将第一个向量的每一位逆序赋给第二个向量，从而实现向量的翻转。解答代码如下：

```
module top_module(
    input  [7:0]          in,
    output [7:0]          out
);
    assign out[7:0] = {in[0], in[1], in[2], in[3], in[4], in[5], in[6], in[7]};
endmodule
```

题目 2：最大值问题

1. 要将 always 赋值语句改为 assign 赋值语句，需将变量修改为 wire 类型，再利用一个条件运算符判断谁是较大者即可实现功能。求解代码如下：

```
module MAX2 (
    input  [7:0]         num1, num2,
    output [7:0]         max
);
    assign max = (num1 > num2) ? num1 : num2;
endmodule
```

2. 从求两个数中的较大者推广到求三个数中的最大值，只需先利用 if 判断语句得出任意两个数中的较大者，再将所得数与第三个数比较即可。解答代码如下：

```
module MAX3 (
    input       [7:0]         num1, num2, num3,
    output reg  [7:0]         max
);
    always @(*) begin
        if (num3 > num1)
            max = (num3 > num2) ? num3 : num2;
        else
            max = (num1 > num2) ? num1 : num2;
    end
endmodule
```

3. 通过基于名字的端口关联例化 MAX2 得到 num1,num2 中的较大者，再次例化 MAX2 得到三个数中的最大者。求解代码如下：

```
module MAX3 (
    input   [7:0]         num1, num2, num3,
    output  [7:0]         max
);
    wire [7:0] bigger_of_two;

    MAX2 max_1 (
        .num1(num1),
        .num2(num2),
        .max(bigger_of_two)
    );

    MAX2 max_2 (
        .num1(bigger_of_two),
        .num2(num3),
        .max(max)
    );

endmodule
```

题目 3：1的个数

对于一个位宽为3的信号，将每一位的数值相加，得到的就是1的个数。解答代码如下：

```
module Count4Ones(
    input       [2:0]         in,
    output reg  [1:0]         out
);
```

```
    always @(*) begin
        out = in[0] + in[1] + in[2];
    end
endmodule
```

【选择性必做部分】

选择的是 **题目 2：Verilog 运算符**，根据运算符的运算规则，可以得到当 a = 8'b0011_0011, b = 8'b1111_0000，各个变量的值的结果如下：

```
c = 8'b0011_0000
d = 8'b0000_0001
e = 8'b1100_0011
f = 8'b1100_1100
g = 8'b0110_0001
h = 8'b0001_1110
i = 8'b0000_0000
j = 8'b1111_0000
k = 8'b0100_0011
l = 8'b0000_0000
```

四、总结

通过本次实验，接触了一门新的语言—— Verilog 语言，初步学习、了解并掌握了 Verilog 的基本语法知识，学会编写简单的 Verilog 代码，并配置好了本地的 VSCode 环境，能够更加方便地编写 Verilog 代码。同时还认识到，Verilog 编程语言得到的是逻辑电路，而非完全顺序执行的代码，它有着对应的逻辑电路结构，编写 Verilog 代码的过程也是描述电路的过程。